

实验 2 第 1 阶段 实验报告

李甘 2023202296

说明：由于不知道应该修改什么文件，我使用了 Gemini 为我提供帮助。对话记录可以在 <https://gemini.google.com/share/e8cce2888487> 看到。

实现 k2023202296_delay_run 系统调用

在 include/prototypes.h 中创建声明：

```
extern syscall k2023202296_delay_run(int, void*, int, ...);
```

在 system/sleep.c 中实现该系统调用，思路为用 create 创建一个进程，这个进程的内容为先 sleep 然后执行 func：

```
/*-----  
 * delay_executor - （内部函数）代理进程执行体，先休眠后执行目标函数  
 *-----  
 */  
local void u2023202296_delay_executor(int seconds, void (*func)  
(int,int,int,int,int),  
                                     int a1, int a2, int a3, int a4, int a5)  
{  
    sleep(seconds);  
    (*func)(a1, a2, a3, a4, a5);  
}  
  
/*-----  
 * delay_run - 异步延时调用指定的函数，不阻塞当前进程  
 *-----  
 */  
syscall k2023202296_delay_run(int seconds, void *func, int nargs, ...)  
{  
    int32 i;  
    va_list ap;  
    int args[5] = {0};  
    if (seconds < 0 || func == NULL)  
        return SYSERR;  
    va_start(ap, nargs);  
    for (i = 0; i < nargs && i < 5; i++)  
        args[i] = va_arg(ap, int);  
    va_end(ap);  
    pid32 pid = create(u2023202296_delay_executor, 1024, 20, "delay_proc",  
7,  
                    seconds, func, args, args, args, args, args);  
    if (pid == SYSERR)  
        return SYSERR;
```

```
    resume(pid);  
    return OK;  
}
```

实现 lab2 命令以测试 k2023202296_delay_run

先在 include/shprototypes.h 中声明这个命令的主函数：

```
/* in file xsh_lab2.c */  
extern shellcmd u2023202296_xsh_lab2 (int32, char *[]);
```

然后在 shell/shell.c 的 cmdtab 中注册这个命令：

```
{"lab2",    FALSE,    u2023202296_xsh_lab2},
```

最后在 shell/xsh_lab2.c 中实现这个命令，最初没有最后的 sleep(4);，在 k2023202296_delay_run 中的 func 被执行前命令进程就退出了，通过在最后加上 sleep 语句可以让主进程在 func 执行完成后再退出：

```
#include <xinu.h>  
#include <string.h>  
#include <stdio.h>  
  
void u2023202296_delay_test(int n) {  
    printf("delay_test: %.2d\n", clktime, count1000);  
    printf("u2023202296_delay_test called with n = %.2d\n", n);  
}  
  
shellcmd u2023202296_xsh_lab2(int nargs, char *args[]) {  
    printf("xsh_lab2(1): %.2d\n", clktime, count1000);  
    k2023202296_delay_run(1, u2023202296_delay_test, 1);  
    printf("xsh_lab2(1): %.2d\n", clktime, count1000);  
    k2023202296_delay_run(2, u2023202296_delay_test, 2);  
    printf("xsh_lab2(1): %.2d\n", clktime, count1000);  
    k2023202296_delay_run(3, u2023202296_delay_test, 3);  
    sleep(4);  
    printf("2023202296 李甘\n");  
    return 0;  
}
```

编译和测试

编译：

```
$ cd compile
$ make clean
$ make
```

运行：

```
qemu-system-i386 -nographic -serial mon:stdio -kernel xinu.elf
```

The screenshot shows a Visual Studio Code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project named 'COLLEGE-XINU-LAB' with files like 'config', 'DESCRIPTION', 'lex.yy.c', 'Makefile', 'y.tab.c', 'device', 'doc', 'lab1', 'lab2', 'include', 'out', and 'timelime'. The 'lab2' directory is selected, showing 'lab1-report.md' and 'lab2-requirement.pdf'. The terminal window shows the output of the 'lab2' command, which is a report for '实验 2 第 1 阶段 实验报告' (Experiment 2 Stage 1 Report). The report includes a title, a description, and a list of files. The terminal also shows the output of the 'qemu-system-i386 -nographic -serial mon:stdio -kernel xinu.elf' command, which is the command used to run the XINU kernel. The terminal output shows the SeaBIOS boot process, including the IPXE boot loader, the XINU kernel version (4), and the memory layout. The terminal also shows the XINU logo and the 'Welcome to Xinu!' message.

```
doc > lab2 > lab1-report.md > ** # 实验 2 第 1 阶段 实验报告 > ** ## 编译和测试
1 # 实验 2 第 1 阶段 实验报告
93 ## 编译和测试
102
103 运行：
104
105 ````plain
106 qemu-system-i386 -nographic -serial mon:stdio -kernel xinu.elf
107
108
109
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

qemu-system-i386 - compile

SeaBIOS (version Arch Linux 1.17.0-2-2)

iPXE (http://ipxe.org) 00:03:0 C900 PCI2.10 PnP PMM+06FD3EE0+06F33EE0 C900

Booting from ROM..

Xinu for QEMU -- version #4 (nictheboy) Tue Apr 14 17:35:00 CST 2026

132829184 bytes of free memory. Free list:

[0x00131000 to 0x07FDDFFF]

59197 bytes of Xinu code.

[0x00100000 to 0x0010E73C]

127692 bytes of data.

[0x00111200 to 0x001304CB]

XINU

Welcome to Xinu!

xsh \$

运行 lab2 命令：

```
xsh $ lab2
```

The screenshot shows a Visual Studio Code editor window with the following components:

- Explorer Panel:** Displays the file structure of the 'COLLEGE-XINU-LAB' project. The 'doc' directory is expanded, showing 'lab1' and 'lab2'. 'lab2' is further expanded, showing 'lab1-report.md' (selected), 'lab2-requirement.pdf', and 'stage1-flig1.png'.
- Editor Panel:** Shows the content of 'lab1-report.md'. The text includes a title '实验 2 第 1 阶段 实验报告', a subtitle '编译和测试', and a code block for 'xsh \$ lab2'.
- Terminal Panel:** Displays the output of the 'xsh \$ lab2' command. The output shows the booting process of XINU, including memory statistics and the XINU logo.

```
doc > lab2 > lab1-report.md > abc # 实验 2 第 1 阶段 实验报告 > abc ## 编译和测试
1 # 实验 2 第 1 阶段 实验报告
93 ## 编译和测试
113 plain
114 xsh $ lab2
115 ...
116
117
118
119
```

Booting from ROM..

Xinu for QEMU -- version #4 (nictheboy) Tue Apr 14 17:35:00 CST 2026

132829184 bytes of free memory. Free list:

[0x00131000 to 0x07FDDFFF]

59197 bytes of Xinu code.

[0x00100000 to 0x0010E73C]

127692 bytes of data.

[0x00111200 to 0x001304CB]

XINU

Welcome to Xinu!

xsh \$ lab2

xsh_lab2(1): 135.229

xsh_lab2(1): 135.231

xsh_lab2(1): 135.232

delay_test: 136.231

u2023202296_delay_test called with n = 134078372

delay_test: 137.232

u2023202296_delay_test called with n = 134078372

delay_test: 138.233

u2023202296_delay_test called with n = 134078372

2023202296 季甘

xsh \$

可以看到，命令的执行结果符合预期。